

Logging in Python for MLOps



PRESENTED BY:

ADITYA RANJAN

DIPTANSHU KUMAR

GARV AGARWALLA

GNANENDRA NAIDU N



Importance of Logging in MLOps

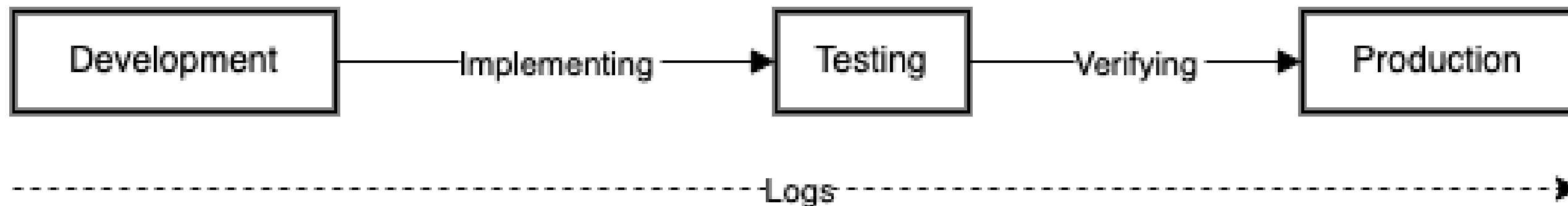
Debugging

Logs provide critical insights, enabling teams to **identify and resolve** issues swiftly in production environments.

Reproducibility

Consistent logging practices ensure that experiments are **easily replicated**, fostering reliability in machine learning outputs.

MLOps - Hierarchies of Logging Implementation



Logging vs Metrics

Logging provides a **narrative context** for understanding system behavior. Unlike simple print statements, logs capture events with timestamps and severity levels. This facilitates deeper insights into application flow and debugging. Logs are essential for post-mortem analysis, allowing developers to trace back through operations leading to an issue, thus addressing failures effectively. They are a **core component** of observability in machine learning workflows.

Metrics, on the other hand, offer a **quantitative overview** of system performance. They aggregate specific data points like error rates or response times to give a snapshot of system health. While they excel in monitoring trends, metrics lack the granularity that logs provide for diagnostics. It's crucial to understand that metrics and logs serve distinct yet complementary purposes in an MLOps framework, each enhancing the visibility of the ML lifecycle.

Logging

- Records events happening in the system
- Provides detailed messages
- Includes timestamps
- Shows log levels (INFO, ERROR, etc.)
- Helps in debugging problems
- Explains why something failed
- Captures errors and warnings
- Useful for troubleshooting
- Generates large amounts of data
- Mostly used by developers and engineers

Metrics

- Measures numerical values
- Shows system performance
- Tracks values over time
- Used for monitoring health
- Helps detect performance degradation
- Used to set alerts and thresholds
- Represents aggregated data
- Easy to visualize using graphs
- Generates less data than logs
- Used by operations and monitoring teams



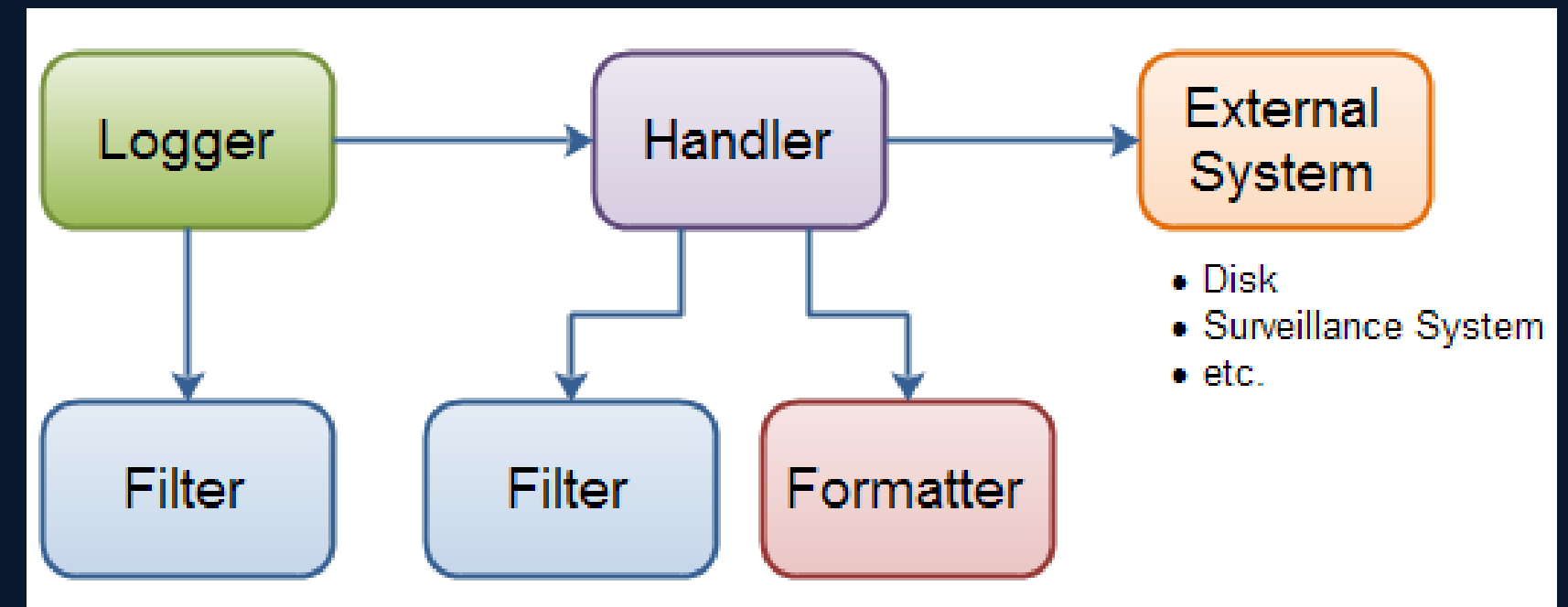
Understanding Logging Components

Logger

The **Logger** is responsible for creating log entries, managing log levels, and routing logs to appropriate handlers.

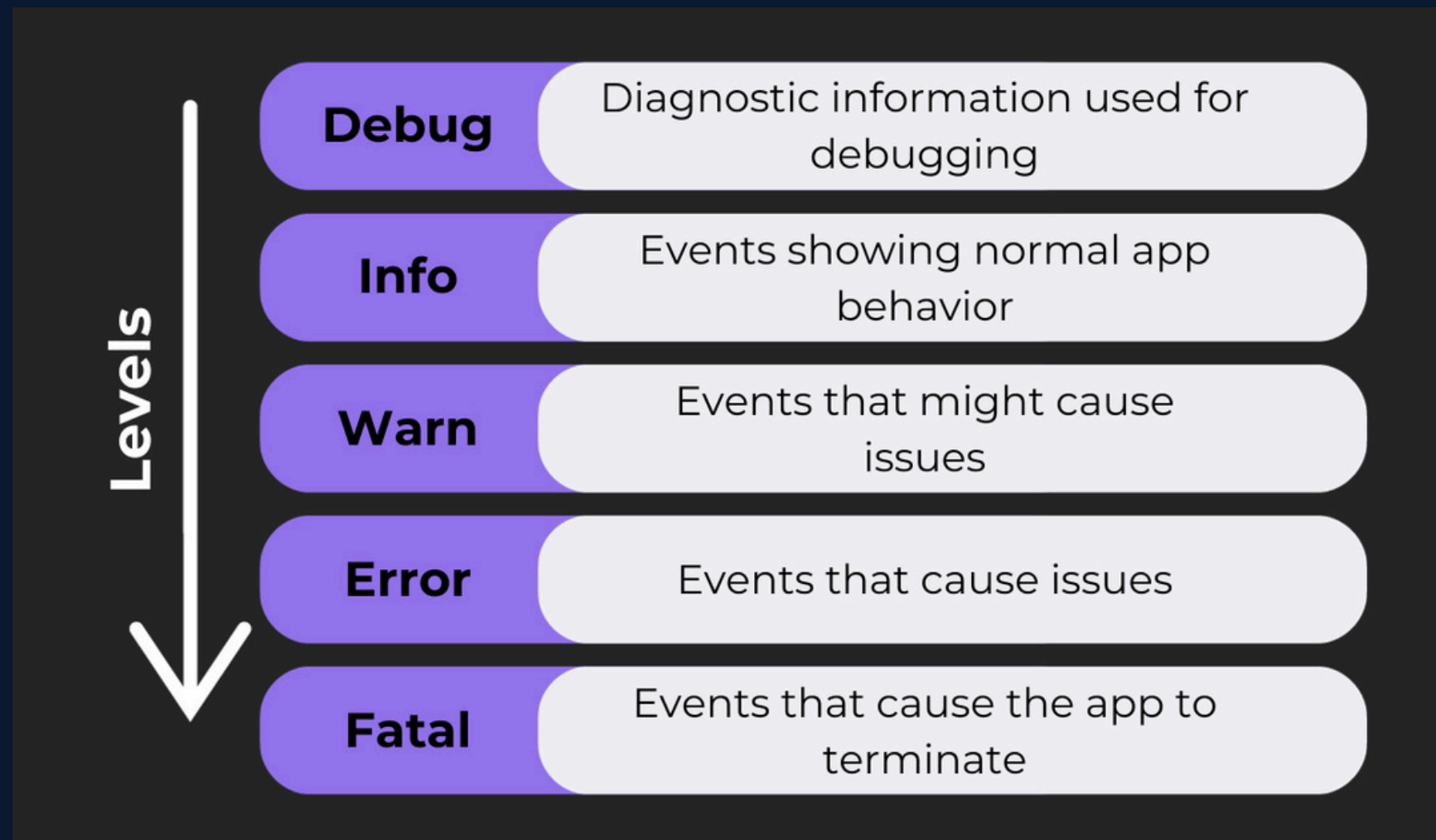
Handler

A **Handler** processes log messages, sending them to different outputs like files, consoles, or external systems.



- **Logger:** Captures log messages from the application and passes them forward.
- **Filter & Formatter :** Filters decide which logs to keep, and the formatter defines how logs look (time, level, message).
- **Handler → External System :** Handlers send the processed logs to destinations like files, disks, or monitoring systems.

Logging Levels in MLOps



- **DEBUG** logs give detailed internal information, mainly used during development to understand how the program is working.
- **INFO** logs confirm that the application is running normally and important steps are completed successfully.
- **WARNING** logs indicate something unexpected that might cause a problem in the future, but the program continues to run.
- **ERROR** logs show that a serious issue occurred and part of the application failed to work correctly.
- **CRITICAL** logs represent severe failures that can cause the entire application to stop.



Logging Configuration

Scalability and Control for ML Projects

Effective logging configuration is essential for managing multiple environments. Using **environment-based settings** allows seamless transitions between development and production, ensuring logs are structured and relevant.

```
logging.basicConfig(  
    level=logging.INFO,  
    format="%(asctime)s - %(levelname)s - %(message)s",  
    datefmt="%Y-%m-%d %H:%M:%S",  
    filename="app.log",  
    filemode="a"  
)
```

level → Controls the severity of logs captured (DEBUG, INFO, WARNING, ERROR)

format → Defines how the log message is displayed

datefmt → Specifies the timestamp format

handlers → Determines where logs are sent (console, file)

filename → Stores logs in a file instead of the console

Basic Logging Example

Implementing logging in Python scripts

This example demonstrates the use of **logging.basicConfig** to configure logging settings, allowing users to replace print statements with structured log messages for better observability and debugging.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s")

x = 10 / 2
logging.info(f"Computation successful, result = {x}")
```

```
2025-12-22 15:03:52,811 - INFO - Computation successful, result = 5.0
```

Why Logging is Useful

- Captures events automatically during program execution
- Adds timestamps to track when actions occur
- Uses severity levels to classify messages
- Helps identify and debug issues quickly

2025-12-22 15:03:52,811 → Exact time when the event occurred

INFO → Log level indicating normal program execution

Computation successful, result = 5.0 → Message describing what happened



Logging in Data Pipelines

```
import logging
import pandas as pd

logging.basicConfig(level=logging.INFO)
data = pd.DataFrame({"name": ["A", "B", None], "age": [20, None, 25]})

logging.info("Starting data pipeline")

if list(data.columns) != ["name", "age"]:
    logging.error("Schema validation failed")
else:
    logging.info("Schema validation passed")

if data.isnull().any().any():
    logging.warning("Missing values detected in dataset")

logging.info("Data pipeline completed")
```

Schema Checks

Verifying data schemas ensures that data meets expected formats, preventing downstream errors in ML processes.

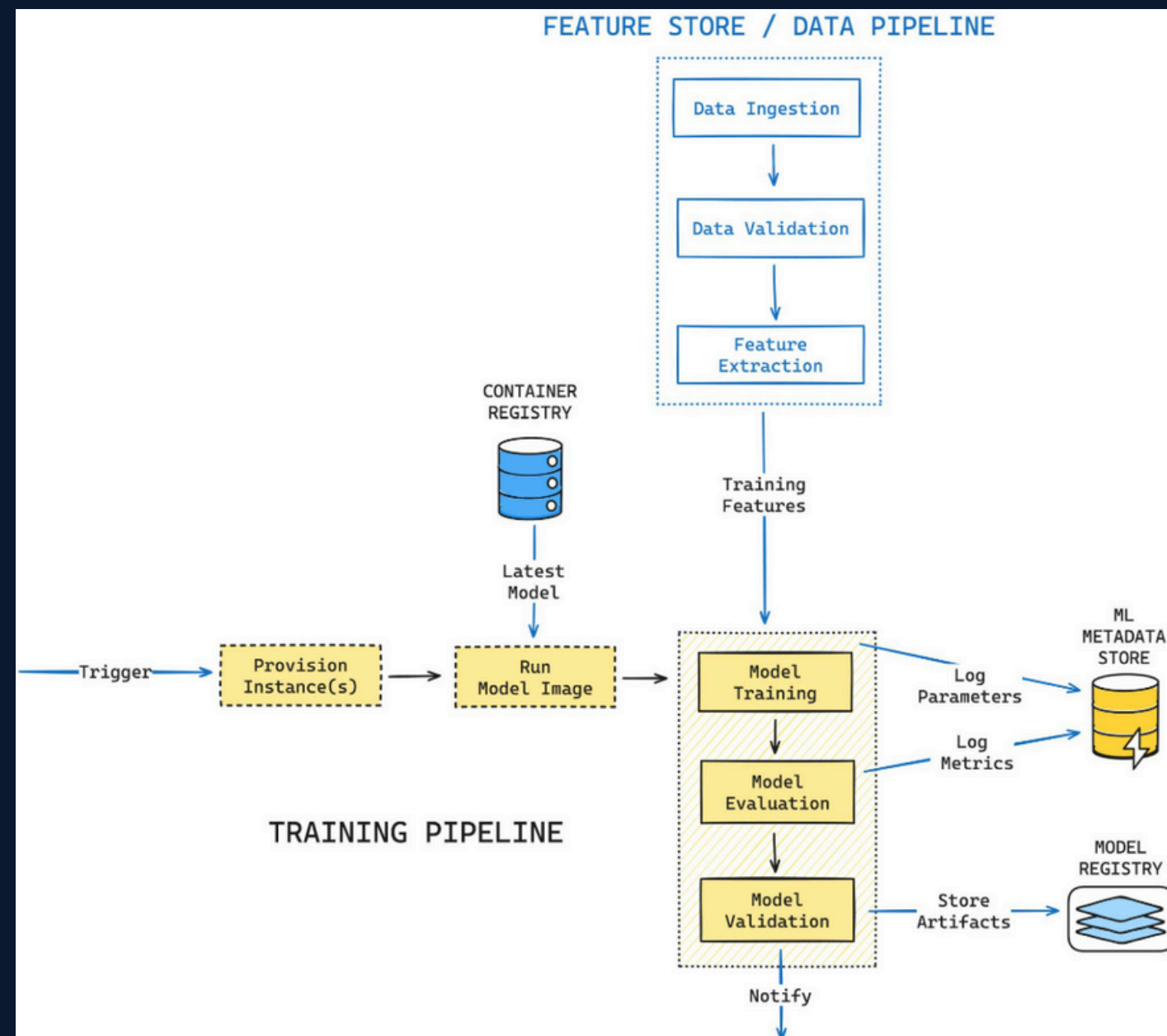
Missing Values

Logging missing values allows teams to address data quality issues proactively, enhancing model reliability and performance.

```
INFO:root:Starting data pipeline
INFO:root:Schema validation passed
WARNING:root:Missing values detected in dataset
INFO:root:Data pipeline completed
```




Logging During Model Training



Hyperparameters

Effective logging of hyperparameters ensures that every model's configuration is easily reproducible and auditable.

Epochs

Logging epochs provides insights into training progress and helps identify convergence issues during model development.



Comparing Logs and Metrics

Logs

Logs provide essential context, capturing detailed information about model performance and system behavior during experiments.

Logs

- Record detailed events
- Capture what happened and why
- Used for debugging & error analysis
- Example: prediction errors, data issues

Metrics

Metrics summarize key performance indicators, allowing for quick evaluations of model effectiveness and operational health in production.

Metrics

- Track numerical performance values
- Show how well the model is performing
- Used for monitoring & alerts
- Example: accuracy, latency, loss

Logs → “*Why did it fail?*”

Logs explain alerts

Metrics → “*How often does it fail?*”

Metrics trigger alerts



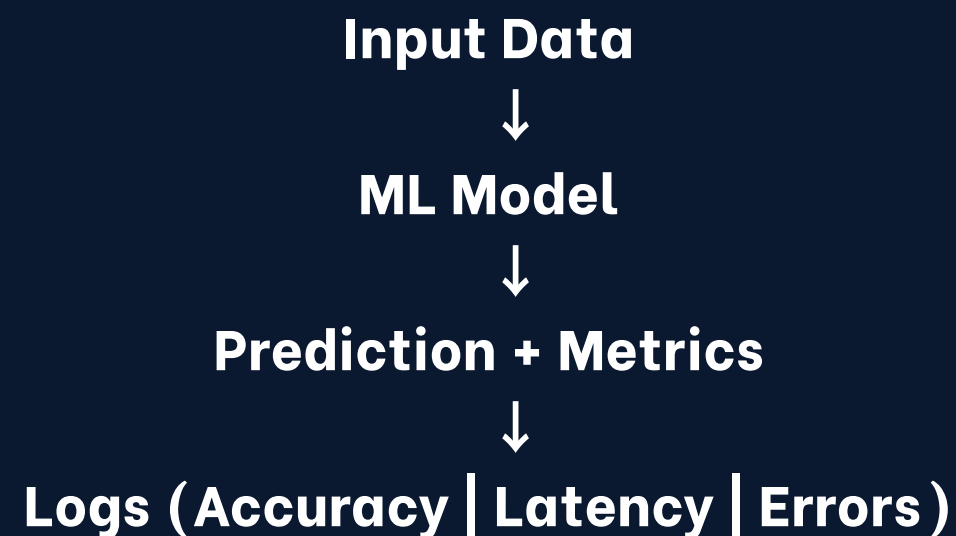
Monitoring Model Performance

Prediction Distribution

Monitoring **prediction distribution** allows for identifying shifts in data patterns affecting model performance over time.

Drift Signals

Detecting **drift signals** is essential for ensuring model reliability and maintaining accuracy in changing environments.



What do we log?

- Predictions & confidence scores
- Accuracy / loss values
- Inference time
- Errors or data drift warnings

Why logging helps?

- Tracks model behavior over time
- Detects performance degradation
- Helps debug wrong predictions
- Enables monitoring in production



Logging in ML Pipelines

Orchestration Logs

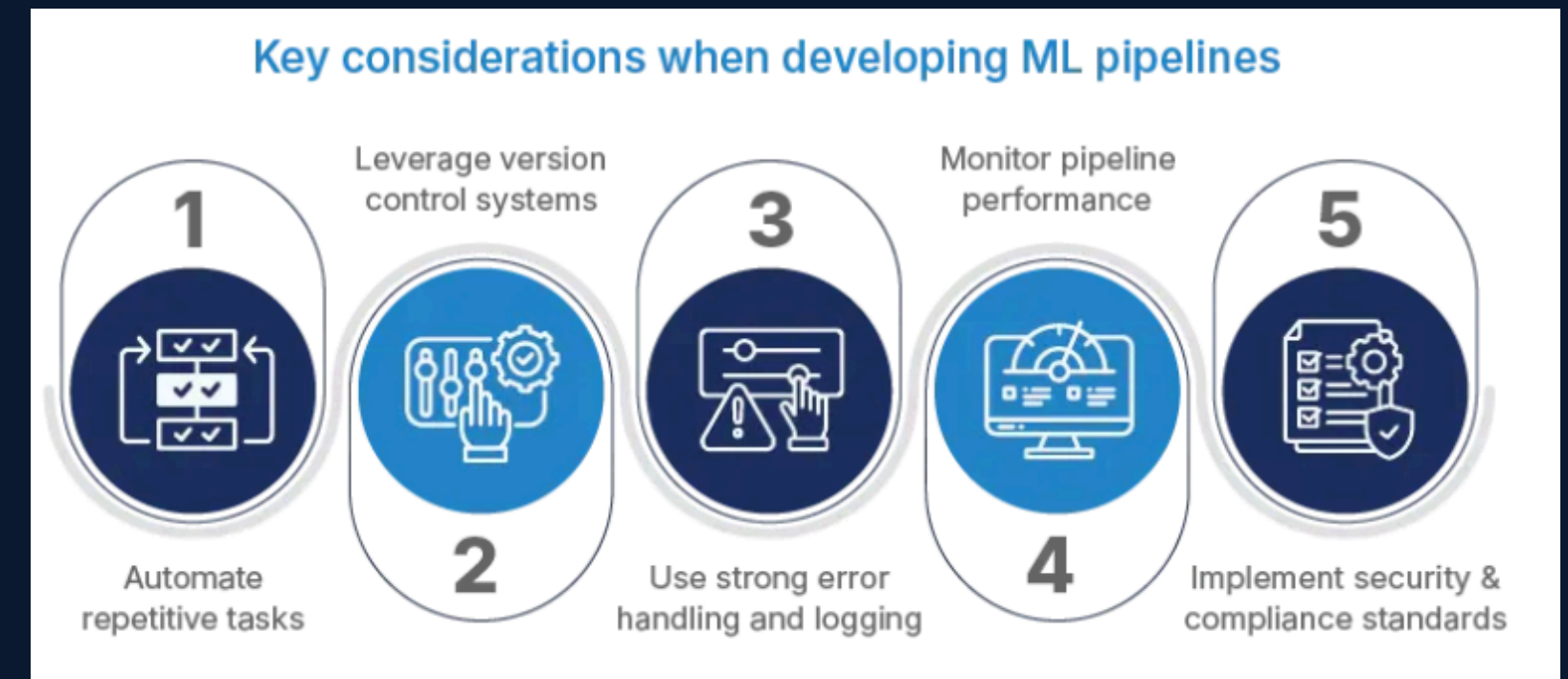
Orchestration platforms like Airflow and Kubeflow generate logs that offer critical insights into pipeline execution.

Retry Mechanisms

Effective retry mechanisms enhance pipeline resilience, ensuring successful task completion despite transient failures.

Best Practices

- **Use Appropriate Log Levels:** Reserve DEBUG for verbose development, INFO for operational flow, and ERROR/CRITICAL for failures.
- **Provide Rich Context:** Include unique IDs (correlation IDs) and specific metadata (e.g., “Failed to process batch ID 123 due to missing feature X”).
- **Avoid Sensitive Data:** Never log PII, passwords, or API keys to ensure security and compliance.
- **Centralize Logs:** In distributed systems, send all logs to a central platform for unified monitoring and faster troubleshooting.





Distributed & Production Logging

Distributed Logging

Efficiently managing logs across **multi-GPU systems** requires a centralized approach to simplify analysis and troubleshooting.

Kubernetes Logging

Leveraging **Kubernetes** for container orchestration enables streamlined logging strategies that enhance observability and fault detection.

Data & Model Stage Logs

- Log data validation results (schema, missing values)
- Record model training progress and checkpoints
- Helps trace issues across pipeline stages

Error & Alert Logs

- Capture exceptions and system errors
- Enable quick debugging and alerting
- Reduce pipeline downtime